

```
>>> num
'forty two'
```

This code doesn't crash when 'forty two' is passed to `int()` because the `ValueError` that `int()` raises is handled by the `except` statement. But doing nothing in response to an error might be worse than a crash. Programs crash so they don't continue to run with bad data or in incomplete states, which could lead to even worse bugs later on. Our code doesn't crash when nondigit characters are entered. But now the `num` variable contains a string instead of an integer, which could cause issues whenever the `num` variable gets used. Our `except` statement isn't handling errors so much as hiding them.

Handling exceptions with poor error messages is another code smell. Look at this example:

```
>>> try:
...     num = input('Enter a number: ')
...     num = int(num)
... except ValueError:
...     print('An incorrect value was passed to int()')
...
Enter a number: forty two
An incorrect value was passed to int()
```

This code doesn't crash, which is good, but it doesn't give the user enough information to know how to fix the problem. Error messages are meant to be read by users, not programmers. Not only does this error message have technical details that a user wouldn't understand, such as a reference to the `int()` function, but it doesn't tell the user how to fix the problem. Error messages should explain what happened as well as what the user should do about it.

It's easier for programmers to quickly write a single, unhelpful description of what happened rather than detailed steps that the user can take to fix the problem. But keep in mind that if your program doesn't handle all possible exceptions that could be raised, it's an unfinished program.